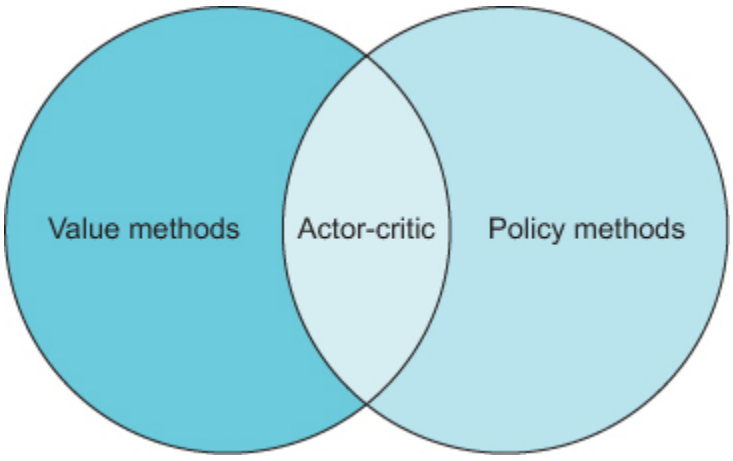
reward +10, but then we'd use our value look-up table and see that on average we get +4 after visiting this state, so the advantage of action 1 given this state is 10 – 4 = +6. This means that when we took action 1, we got a reward that was significantly better than what we expected based on past rewards from that state, which suggests that it was a good action. Compare this to the case where we take action 1 and receive reward +10 but our value look-up table says we expected to see +15, so the advantage is 10 – 15 = –5. That suggests this was a relatively bad action despite the fact that we received a reasonably large positive reward.

Rather than using a look-up table, we will use some sort of parameterized model, such as a neural network that can be trained to predict expected rewards for a given state. So we want to simultaneously train a policy neural network and a state-value or action-value neural network.

Algorithms of this sort are called *actor-critic* methods, where "actor" refers to the policy, because that's where the actions are generated, and "critic" refers to the value function, because that's what (in part) tells the actor how good its actions are. Since we're using $R - V(S)$ to train the policy rather than just $V(S)$, this is called *advantage actor-critic* (figure 5.2).

Figure 5.2. Q-learning falls under the category of value methods, since we attempt to learn action values, whereas policy gradient methods like REINFORCE directly attempt to learn the best actions to take. We can combine these two techniques into what's called an actor-critic architecture.



Note